Viridis Research — certification record

Run `runs/2026-11-03_P1` 2026-11-03 project P1, task P1-T002 engine `12e43010959a`
GitHub run: `https://github.com/jdhart81/viridis-engine/actions/runs/41560379915`

Toward a machine-checked raster bridge for Trees Cannot Cut: the bit-quad formula and proposed statements for single-pixel insertion and the enclosure identity
Justin Hart, Viridis LLC

Labels awarded by the verifier

Claim	Label	Claim text (as submitted)
C1	**ASSUMED**	empirical: An independent brute-force computation (DFS component labelling on a padded grid, direct V-E+F count) finds no counterexample to the proposed P1.Bridge identity enc(B) = C8(B) - euler(B) on all 65,536 subsets of a 4x4 box and on 3,000 random rasters up to 12x12 (seed 20261103), and none to the proposed P1.Insertion invariance on 10,496 insertions covering all 256 configurations of the 8-neighbourhood of the inserted pixel, each alone and in 40 random contexts.
C2	**ASSUMED**	conjecture: The proposed statements P1.Insertion and P1.Bridge are true: for every finite raster B in Z^2 under (8,4) connectivity, the set of finite 4-components of Z^2 \ B is finite and its size equals C8(B) - euler(B), and inserting any pixel preserves enclosed - C8 + euler.

Other checks

- Lean toolchain leanprover/lean4:v4.28.0, Mathlib 8f9d9cff6bd7; kernel replay by `leanchecker`.
- References: 6 entries, 6 DOIs, 6 resolved, 0 unresolved, 0 unchecked.
- Agent paper: compiled.
- Statement proposals: 2.
- P1.Insertion: rejected: object file '/out/ViridisStatements/P1/BitQuad.olean' of module ViridisStatements.P1.BitQuad does not exist
- P1.Bridge: rejected: object file '/out/ViridisStatements/P1/BitQuad.olean' of module ViridisStatements.P1.BitQuad does not exist

How to read this record

ASSUMED: not verified (the default). **PROVEN**: a Lean 4 proof of a statement sealed before the attempt, compiled with the pinned toolchain, matched definitionally to the sealed statement, using only the axioms propext, Classical.choice and Quot.sound, and replayed through the kernel. **TESTED/REPLICATED**: reproduced by the verifier (not yet enabled). **ENDORSED/ADOPTED**: granted by people. Novelty is never a label. **TRIVIAL** marks a sealed statement that one of the engine's probe tactics already closes.

This paper was drafted by an AI research agent (Claude, by Anthropic) running the Viridis research engine; formal proof search may use Aristotle (Harmonic). Justin Hart directs the research program. The labels on this page were assigned by an automated verifier, not by the agent, and only they carry certification. Claims in the paper that are not listed here as PROVEN are ASSUMED.

Reproduce everything from `REPRODUCE.md` in the run folder; the run's files are listed with hashes in `MANIFEST.sha256`.

Toward a machine-checked raster bridge for *Trees Cannot Cut*: the bit-quad formula and proposed statements for single-pixel insertion and the enclosure identity

Justin Hart
Viridis LLC

3 November 2026

Abstract

For a finite set $B \subset \mathbb{Z}^2$ of barrier pixels, read with 8-connected barriers and a 4-connected complement, the expected discrete form of Alexander duality is the identity $\mathrm{enc}(B) = C_8(B) - \chi(K(B))$, where $\mathrm{enc}(B)$ is the number of finite 4-components of $\mathbb{Z}^2 \setminus B$, C_8 counts 8-components of B and $\chi(K(B)) = V - E + F$ is the Euler characteristic of the union of the closed pixel squares. This identity would replace the topological hypothesis `FaceCount` that the machine-checked core of *Trees Cannot Cut* currently assumes. The project climbs a three-rung ladder. For the first rung (L1), Gray's bit-quad formula $4\chi = Q_1 - Q_3 - 2Q_D$, we gave a Lean 4 proof in an earlier session by double counting over the vertices of $K(B)$. This version adds precise Lean statements proposed for sealing for the other two rungs: `P1.Insertion` (L2: inserting any pixel leaves enclosed $- C_8 + \chi$ unchanged, with no hypothesis on its 3×3 neighbourhood) and `P1.Bridge` (L3: the identity above, together with finiteness of the set of enclosed components). Both use Mathlib's graph library on all of $\mathbb{Z}^2$ and define a component as enclosed exactly when it is finite. Both reuse χ from the sealed L1 module. Each module checks the project's controls (the 3×3 ring, the solid 2×2 block, and the four-pixel diamond that is enclosing only under (8,4) connectivity) by `decide` on a computable bounded-box model. It also gives Lean proofs of the same control values for the non-computable definitions that the statements actually use. An independent brute-force computation finds no counterexample to L3 on all 65,536 subsets of a 4×4 box or on 3,000 random rasters, and none to L2 across all 256 neighbourhood configurations. L2 and L3 themselves remain unproved.

1 Introduction

Hart's *Trees Cannot Cut* [3] argues that fuel breaks and habitat corridors split a forest landscape into separate compartments only through their cycles: an acyclic corridor leaves the fuel matrix connected. The graph theory behind this claim is machine-checked, but the step from topology to graph theory is carried as an explicit hypothesis (`FaceCount`), which says that the number of compartments equals the circuit rank plus one. The manuscript justifies it by Alexander duality. Mathlib [6] has neither Alexander duality nor a planar Euler formula in a form that applies here.

Fire and fuel-break data usually come as rasters, so project P1 aims to discharge the hypothesis in the raster setting. For a finite barrier raster B with 8-connected barriers and a 4-connected complement, the target identity is

$$\#\{\text{enclosed 4-components of } \mathbb{Z}^2 \setminus B\} \;=\; C_8(B) - \chi(K(B)).$$

The proposed route is a ladder of three lemmas. L1 is a local formula for $\chi(K(B))$. L2 says that adding one pixel preserves enclosed $- C_8 + \chi$. L3 is the bridge itself, by induction on $|B|$. The first version of this paper did L1: it gives a Lean proof of Gray's bit-quad formula in its 8-connected form (Sections 3–7). This version adds exact Lean statements, proposed for sealing, for L2 and L3 (Section 8). It also adds machine-checked sanity checks on their definitions and computational evidence that the proposed statements are true (Section 9). A proposed statement is a target, not a result: we do not prove L2 or L3 here. L1 matters for the rest of the ladder because it makes the change in χ under a single-pixel insertion a purely local quantity: only the four windows around the new pixel change.

2 Background and related work

Gray [2] introduced locally countable properties of binary images, among them area, perimeter and the Euler number. He showed that the Euler number can be computed from counts of 2×2 "bit-quad" patterns, with different weightings for 4- and 8-connectivity of the foreground. The (8,4) pairing of connectivity for foreground and background goes back to Rosenfeld [5], who showed that with this pairing a simple closed digital curve separates the plane into exactly two components. Its role in digital Jordan-type theorems is surveyed by Kong and Rosenfeld [4]. In the 8-connected reading, two pixels that share only a corner touch in $K(B)$. The two diagonal patterns then contribute -2 to 4χ, because their shared corner is one vertex where four edges meet. Our Lean development uses Lean 4 [1] and Mathlib [6]. We are not aware of an earlier machine-checked proof of the bit-quad formula, but we have not searched exhaustively and make no claim of novelty.

3 Setting: the sealed statement

The statement module `ViridisStatements.P1.BitQuad` fixes the following definitions. A pixel $p = (i, j) \in \mathbb{Z}^2$ stands for the closed square $[i, i+1] \times [j, j+1]$.

- $\mathrm{vertices}(B) = \bigcup_{p \in B} \mathrm{corners}(p)$, where $\mathrm{corners}(i, j) = \{(i, j), (i+1, j), (i, j+1), (i+1, j+1)\}$.
- $\mathrm{hEdges}(B) = \bigcup_{(i,j) \in B} \{(i, j), (i, j+1)\}$ (horizontal edges named by their left end) and $\mathrm{vEdges}(B) = \bigcup_{(i,j) \in B} \{(i, j), (i+1, j)\}$ (vertical edges named by their lower end).
- The Euler characteristic, computed in $\mathbb{Z}$:

$$\chi(B) = |\mathrm{vertices}(B)| - (|\mathrm{hEdges}(B)| + |\mathrm{vEdges}(B)|) + |B|.$$

- The window at a lattice point $v = (x, y)$ consists of four bits, $a = [(x-1, y-1) \in B]$, $b = [(x, y-1) \in B]$, $c = [(x-1, y) \in B]$ and $d = [(x, y) \in B]$, with $n(v) = a + b + c + d$.
- Q_1, Q_3 and Q_D count the vertices v with $n(v) = 1$, with $n(v) = 3$, and with $\{a, d\}$ or $\{b, c\}$ as the only set bits.

Every window that meets B is centred on a vertex of $K(B)$, so counting over the vertices counts every such window exactly once.

Theorem 1 (`P1.BitQuad`)**.** *For every finite $B \subset \mathbb{Z}^2$, $4\,\chi(K(B)) = Q_1(B) - Q_3(B) - 2\,Q_D(B)$.*

4 Proof architecture

Write $\mathcal{V} = \text{vertices}(B)$, and use $[P] \in \{0,1\}$ for the indicator of a proposition P.

Lemma 2 (Edge membership)**.** $(i,j) \in \text{hEdges}(B) \iff (i,j) \in B \vee (i,j-1) \in B$*, and* $(i,j) \in \text{vEdges}(B) \iff (i,j) \in B \vee (i-1,j) \in B$*.*

Lemma 3 (Shift counting)**.** *Let* $S, \mathcal{V} \subset \mathbb{Z}^2$ *be finite and* $s \in \mathbb{Z}^2$*. If* $x + s \in \mathcal{V}$ *for every* $x \in S$*, then* $\sum_{v \in \mathcal{V}} [v - s \in S] = |S|$*.*

Proof. The map $v \mapsto v - s$ is a bijection from $\{v \in \mathcal{V} : v - s \in S\}$ onto S. Its inverse is $x \mapsto x + s$, which lands in $\mathcal{V}$ by hypothesis. □

Lemma 3 does all of the double counting. Every pixel $x \in B$ has its four corners x, $x + (1,0)$, $x + (0,1)$ and $x + (1,1)$ in $\mathcal{V}$. Applying the lemma with the four shifts gives

$$4F = \sum_{v \in \mathcal{V}} n(v).$$

By Lemma 2, both ends x and $x + (1,0)$ of a horizontal edge are corners of the pixel above or below it, and likewise for vertical edges. Write the arms of v as left $[v - (1,0) \in \text{hEdges}]$, right $[v \in \text{hEdges}]$, down $[v - (0,1) \in \text{vEdges}]$ and up $[v \in \text{vEdges}]$. Applying the lemma once more for each arm gives

$$2E = \sum_{v \in \mathcal{V}} \text{arms}(v), \qquad V = \sum_{v \in \mathcal{V}} 1.$$

Hence $4\chi = \sum_{v \in \mathcal{V}} \big(4 - 2\,\text{arms}(v) + n(v)\big)$. Lemma 2 expresses each arm through the window bits. The left arm is $a \vee c$, the right arm $b \vee d$, the down arm $a \vee b$ and the up arm $c \vee d$. The summand therefore depends only on (a,b,c,d), and Table 1 evaluates it. Every vertex is a corner of some pixel, which must be one of the four window cells, so the empty pattern never occurs. On the other 15 patterns the summand equals $[n = 1] - [n = 3] - 2[\text{diagonal}]$. Summing over $\mathcal{V}$ proves Theorem 1.

Pattern class	#patterns	n	arms	$4 - 2\,\text{arms} + n$
empty (cannot occur)	1	0	0	4
one pixel	4	1	2	1
adjacent pair	4	2	3	0
diagonal pair	2	2	4	−2
three pixels	4	3	4	−1
full	1	4	4	0

Table 1: The local summand on the 16 window patterns.

5 The Lean development

The candidate file `proofs/P1.BitQuad.lean` (145 lines) follows the architecture above.

- `mem_vertices`, `mem_corners`, `mem_hEdges` and `mem_vEdges` unfold the membership predicates (Lemma 2).

- `card_shift` proves Lemma 3 with `Finset.card_bij`. Its corollary `sum_shift` restates it as a sum of $\{0,1\}$-valued integer indicators via `Finset.sum_boole`.
- In `main`, the eight instances of `sum_shift` (four for faces, two for each edge direction) and the three expressions of Q_1, Q_3 and Q_D as indicator sums reduce the goal to a single `Finset.sum_congr`. The pointwise goal is closed by `by_cases` on the four window memberships followed by `simp`, which also uses the fact that the window at a vertex is non-empty.

The brief suggested evaluating the 16 patterns with `decide`. That proved unnecessary: once the window memberships are case-split, `simp` closes all 16 branches directly. The file uses no `decide` on large objects and no `native_decide`. The project checker (`viridis-lean-check`) reports that the file compiles under Lean `v4.28.0` with the pinned Mathlib, matches the sealed statement, depends only on `propext`, `Classical.choice` and `Quot.sound`, and replays through the kernel. Whether the claim is accepted is decided by the project's independent verifier; see its certification page. The complete file follows.

```
import Mathlib
import ViridisStatements.P1.BitQuad

namespace Candidate

open ViridisStatements.P1.BitQuad

/-- Membership in ‘vertices‘. -/
theorem mem_vertices (B : Finset Pixel) (v : ℤ × ℤ) :
    v ∈ vertices B ↔ ∃ p ∈ B, v ∈ corners p := by
  unfold vertices
  simp only [Finset.mem_biUnion]

theorem mem_corners (p v : ℤ × ℤ) :
    v ∈ corners p ↔ v = (p.1, p.2) ∨ v = (p.1 + 1, p.2) ∨ v = (p.1, p.2 + 1) ∨
      v = (p.1 + 1, p.2 + 1) := by
  unfold corners
  simp only [Finset.mem_insert, Finset.mem_singleton]

/-- A horizontal edge is present iff the pixel above or below it is in ‘B‘. -/
theorem mem_hEdges (B : Finset Pixel) (h : ℤ × ℤ) :
    h ∈ hEdges B ↔ (h.1, h.2) ∈ B ∨ (h.1, h.2 - 1) ∈ B := by
  unfold hEdges
  simp only [Finset.mem_biUnion, Finset.mem_insert, Finset.mem_singleton]
  constructor
  · rintro ⟨p, hp, rfl | rfl⟩
    · left; simpa using hp
    · right; simpa using hp
  · rintro (h1 | h1)
    · exact ⟨_, h1, Or.inl rfl⟩
    · exact ⟨_, h1, Or.inr (by simp)⟩

/-- A vertical edge is present iff the pixel right or left of it is in ‘B‘. -/
theorem mem_vEdges (B : Finset Pixel) (e : ℤ × ℤ) :
    e ∈ vEdges B ↔ (e.1, e.2) ∈ B ∨ (e.1 - 1, e.2) ∈ B := by
  unfold vEdges
  simp only [Finset.mem_biUnion, Finset.mem_insert, Finset.mem_singleton]
  constructor
  · rintro ⟨p, hp, rfl | rfl⟩
    · left; simpa using hp
    · right; simpa using hp
```

```
  · rintro (h1 | h1)
    · exact ⟨_, h1, Or.inl rfl⟩
    · exact ⟨_, h1, Or.inr (by simp)⟩

/-- Shift-counting: if every translate `x + (a, b)` of `S` lies in `V`, then
counting the `v ∈ V` with `v - (a, b) ∈ S` gives `|S|`. -/
theorem card_shift (V S : Finset (ℤ × ℤ)) (a b : ℤ)
    (hS : ∀ x ∈ S, (x.1 + a, x.2 + b) ∈ V) :
    (V.filter fun v => (v.1 - a, v.2 - b) ∈ S).card = S.card := by
  apply Finset.card_bij (fun v _ => (v.1 - a, v.2 - b))
  · intro v hv
    exact (Finset.mem_filter.mp hv).2
  · intro v₁ _ v₂ _ h
    simp only [Prod.mk.injEq] at h
    ext <;> omega
  · intro x hx
    refine ⟨(x.1 + a, x.2 + b), Finset.mem_filter.mpr ⟨hS x hx, ?_⟩, ?_⟩
    · simpa using hx
    · simp

theorem sum_shift (V S : Finset (ℤ × ℤ)) (a b : ℤ)
    (hS : ∀ x ∈ S, (x.1 + a, x.2 + b) ∈ V) :
    (∑ v ∈ V, if (v.1 - a, v.2 - b) ∈ S then (1 : ℤ) else 0) = S.card := by
  rw [Finset.sum_boole, card_shift V S a b hS]

theorem main : ViridisStatements.P1.BitQuad.Statement := by
  intro B
  set V := vertices B with hV
  -- inclusion facts
  have hB : ∀ x ∈ B, ∀ a b : ℤ, (a = 0 ∨ a = 1) → (b = 0 ∨ b = 1) →
      (x.1 + a, x.2 + b) ∈ V := by
    intro x hx a b ha hb
    rw [hV, mem_vertices]
    refine ⟨x, hx, ?_⟩
    rw [mem_corners]
    rcases ha with rfl | rfl <;> rcases hb with rfl | rfl <;> simp
  have hH : ∀ x ∈ hEdges B, ∀ a : ℤ, (a = 0 ∨ a = 1) → (x.1 + a, x.2 + 0) ∈ V := by
    intro x hx a ha
    rw [mem_hEdges] at hx
    rcases hx with hx | hx
    · have := hB _ hx a 0 ha (Or.inl rfl)
      simpa using this
    · have := hB _ hx a 1 ha (Or.inr rfl)
      simpa using this
  have hVe : ∀ x ∈ vEdges B, ∀ b : ℤ, (b = 0 ∨ b = 1) → (x.1 + 0, x.2 + b) ∈ V := by
    intro x hx b hb
    rw [mem_vEdges] at hx
    rcases hx with hx | hx
    · have := hB _ hx 0 b (Or.inl rfl) hb
      simpa using this
    · have := hB _ hx 1 b (Or.inr rfl) hb
      simpa using this
  -- the four expressions as sums over V
  have eV : ((V.card : ℤ)) = ∑ v ∈ V, (1 : ℤ) := by simp
  have eF11 := sum_shift V B 1 1 (fun x hx => hB x hx 1 1 (Or.inr rfl) (Or.inr rfl))
  have eF01 := sum_shift V B 0 1 (fun x hx => hB x hx 0 1 (Or.inl rfl) (Or.inr rfl))
  have eF10 := sum_shift V B 1 0 (fun x hx => hB x hx 1 0 (Or.inr rfl) (Or.inl rfl))
  have eF00 := sum_shift V B 0 0 (fun x hx => hB x hx 0 0 (Or.inl rfl) (Or.inl rfl))
  have eH10 := sum_shift V (hEdges B) 1 0 (fun x hx => hH x hx 1 (Or.inr rfl))
```

```
  have eH00 := sum_shift V (hEdges B) 0 0 (fun x hx => hH x hx 0 (Or.inl rfl))
  have eV01 := sum_shift V (vEdges B) 0 1 (fun x hx => hVe x hx 1 (Or.inr rfl))
  have eV00 := sum_shift V (vEdges B) 0 0 (fun x hx => hVe x hx 0 (Or.inl rfl))
  have eQ1 : ((Q1 B : ℕ) : ℤ) = ∑ v ∈ V, if windowCount B v = 1 then (1 : ℤ) else 0 := by
    rw [Finset.sum_boole]; rfl
  have eQ3 : ((Q3 B : ℕ) : ℤ) = ∑ v ∈ V, if windowCount B v = 3 then (1 : ℤ) else 0 := by
    rw [Finset.sum_boole]; rfl
  have eQD : ((QD B : ℕ) : ℤ) = ∑ v ∈ V, if isDiagonal B v = true then (1 : ℤ) else 0 := by
    rw [Finset.sum_boole]; rfl
  have key : 4 * euler B = ∑ v ∈ V,
      ((4 : ℤ) - 2 * ((if (v.1 - 1, v.2 - 0) ∈ hEdges B then 1 else 0)
        + (if (v.1 - 0, v.2 - 0) ∈ hEdges B then 1 else 0)
        + (if (v.1 - 0, v.2 - 1) ∈ vEdges B then 1 else 0)
        + (if (v.1 - 0, v.2 - 0) ∈ vEdges B then 1 else 0))
        + ((if (v.1 - 1, v.2 - 1) ∈ B then 1 else 0)
        + (if (v.1 - 0, v.2 - 1) ∈ B then 1 else 0)
        + (if (v.1 - 1, v.2 - 0) ∈ B then 1 else 0)
        + (if (v.1 - 0, v.2 - 0) ∈ B then 1 else 0))) := by
    simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib, ← Finset.mul_sum]
    rw [eF11, eF01, eF10, eF00, eH10, eH00, eV01, eV00]
    unfold euler
    rw [← hV]
    simp
    ring
  rw [key, eQ1, eQ3, eQD, ← Finset.sum_sub_distrib, Finset.mul_sum,
    ← Finset.sum_sub_distrib]
  apply Finset.sum_congr rfl
  intro v hv
  obtain ⟨x, y⟩ := v
  have hne : (x - 1, y - 1) ∈ B ∨ (x, y - 1) ∈ B ∨ (x - 1, y) ∈ B ∨ (x, y) ∈ B := by
    rw [hV, mem_vertices] at hv
    obtain ⟨p, hp, hc⟩ := hv
    rw [mem_corners] at hc
    rcases hc with h | h | h | h <;> simp only [Prod.mk.injEq] at h <;> obtain ⟨rfl, rfl⟩ := h
    · right; right; right; exact hp
    · right; right; left; simpa using hp
    · right; left; simpa using hp
    · left; simpa using hp
  unfold windowCount isDiagonal bits
  simp only [sub_zero, mem_hEdges, mem_vEdges]
  by_cases ha : (x - 1, y - 1) ∈ B <;> by_cases hb : (x, y - 1) ∈ B <;>
    by_cases hc : (x - 1, y) ∈ B <;> by_cases hd : (x, y) ∈ B <;>
    simp [ha, hb, hc, hd] at hne ⊢

end Candidate
```

6 Results

- We give a Lean proof of Theorem 1, the sealed statement `P1.BitQuad`, for all finite rasters.
- As an independent computational check, which is evidence and not a proof, the script `code/bitquad_check.py` recomputes both sides for all $2^{16} = 65{,}536$ subsets of a 4×4 box. It finds no counterexample (`results/bitquad_4x4.json`). Euler characteristics from -1 to 4 occur in that box.

7 How L1 serves the L2/L3 bridge

Let $p \notin B$ and $B' = B \cup \{p\}$. The windows whose bits differ between B and B' are exactly the four windows centred on the corners of p. By Theorem 1 applied to B and to B', $4(\chi(B') - \chi(B))$ equals the change in $Q_1 - Q_3 - 2Q_D$ over those four windows. That change depends only on the 3×3 neighbourhood of p. This is the "local half" of L2. The non-local half, the change in C_8 and in the number of enclosed 4-components of the complement, still needs a separate argument: a pixel joining two 8-components can close off a new bounded region. L1 contributes nothing there. The negative controls listed in the project context (the solid 2×2 block, whose 8-neighbour graph K_4 has circuit rank 3 yet encloses nothing, and the four-pixel N/S/E/W cross enclosing its centre) show that L2 must be stated in terms of $\chi(K(B))$ and not in terms of the graph circuit rank.

8 Proposed statements for L2 and L3

The two modules `proposals/P1.Insertion.lean` and `proposals/P1.Bridge.lean` are *proposals*. They become targets only if they are sealed, and nothing in this section is a proof of either statement. Proposals may not import one another, so the two modules carry identical copies of the connectivity definitions. Both import the sealed L1 module and use its χ (`euler`) unchanged.

8.1 Definitions and conventions

Definition 4. Let G_4 and G_8 be the graphs on $\mathbb{Z}^2$ in which distinct p, q are adjacent when $|p_1 - q_1| + |p_2 - q_2| = 1$, respectively when $|p_1 - q_1| \le 1$ and $|p_2 - q_2| \le 1$. For a finite B, $C_8(B)$ is the number of connected components of the subgraph of G_8 induced on B. A connected component of the subgraph of G_4 induced on $\mathbb{Z}^2 \setminus B$ is *enclosed* when it is finite, and $\mathrm{enc}(B)$ is the number of enclosed components.

The brief for this task fixes the exterior convention, and the definitions follow it literally. The complement is taken in all of $\mathbb{Z}^2$, never inside a cropping window. "Enclosed" means "finite", so no bounding box enters the statements. In Lean the graphs are built with `SimpleGraph.fromRel`, which symmetrises a relation and removes loops, and components are Mathlib's `ConnectedComponent` of an induced graph (Listing 1).

Listing 1: Definitions and the two proposed statements (each Statement lives in its own module).

```
def G4 : SimpleGraph Pixel :=
  SimpleGraph.fromRel fun p q => |p.1 - q.1| + |p.2 - q.2| = 1
def G8 : SimpleGraph Pixel :=
  SimpleGraph.fromRel fun p q => |p.1 - q.1| ≤ 1 ∧ |p.2 - q.2| ≤ 1
def barrierGraph (B : Finset Pixel) : SimpleGraph (B : Set Pixel) :=
  G8.induce (B : Set Pixel)
def complementGraph (B : Finset Pixel) : SimpleGraph ((B : Set Pixel)ᶜ : Set Pixel) :=
  G4.induce (B : Set Pixel)ᶜ
noncomputable def C8 (B : Finset Pixel) : ℕ :=
  Nat.card (barrierGraph B).ConnectedComponent
def IsEnclosed {B : Finset Pixel} (c : (complementGraph B).ConnectedComponent) : Prop :=
  c.supp.Finite
noncomputable def enclosed (B : Finset Pixel) : ℕ :=
  Nat.card {c : (complementGraph B).ConnectedComponent // IsEnclosed c}

-- P1.Bridge (L3)
def Statement : Prop :=
  ∀ B : Finset Pixel,
```

```
    {c : (complementGraph B).ConnectedComponent | IsEnclosed c}.Finite ∧
      (enclosed B : ℤ) = C8 B - euler B

-- P1.Insertion (L2)
noncomputable def invariant (B : Finset Pixel) : ℤ :=
  (enclosed B : ℤ) - C8 B + euler B
def Statement : Prop :=
  ∀ (B : Finset Pixel) (p : Pixel), p ∉ B → invariant (insert p B) = invariant B
```

L3 (`P1.Bridge`). For every finite B, the set of enclosed components is finite and $\mathrm{enc}(B) = C_8(B) - \chi(K(B))$ in $\mathbb{Z}$. The finiteness clause is deliberate. `Nat.card` returns 0 on an infinite type, so without the clause the equation could be met in a degenerate way. With it, the count means what it says. The clause is true because every component outside the bounding box of B contains a ray.

L2 (`P1.Insertion`). For every finite B and every $p \notin B$, $\mathrm{enc} - C_8 + \chi$ takes the same value on $B \cup \{p\}$ and on B. The statement puts no condition on the 3×3 neighbourhood of p. All 256 configurations of its eight neighbours are therefore covered, in every global context. These include insertions that merge several 8-components, insertions that close off a hole, and insertions that fill one. By L1, $\Delta\chi$ is a function of the neighbourhood (Section 7). ΔC_8 and $\Delta\mathrm{enc}$ are not local, and they are where the proof effort lies.

Relation between the rungs. L3's equation follows from L2 by induction on $|B|$ from the empty raster, where both sides are 0. Conversely, L3's equation gives L2 at once. The two rungs are equivalent given the base case. The split organises the proof around one inductive step; it does not grade the statements by strength.

8.2 Controls

The definitions of C_8 and enc are not decidable, since they quantify over infinite components of $\mathbb{Z}^2$. Each module therefore also contains a computable *bounded-box model*. It flood-fills the complement inside the bounding box of $B \cup \{(0,0)\}$ with a one-pixel margin, starting from the box corner, and counts components by their lexicographically least pixel. The statements do not mention this model, and we do not prove that it agrees with the definitions. Against the box model, `decide` checks the values in Table 2. The `P1.Insertion` module also checks four non-local insertions: closing the ring's top edge, merging the ring's two columns through a corner, completing the diamond, and filling the ring's centre.

raster	C_8	χ	enc	role
empty	0	0	0	base case
3×3 ring	1	0	1	witness
solid 2×2 block	1	1	0	negative control 1 (K_4 has circuit rank 3)
N/S/E/W diamond	1	0	1	negative control 2 (4-adjacency graph has no edges)
diagonal pair	1	1	0	corner contact

Table 2: Control values. All rows are checked by `decide` on the box model. For the first four rows we also give Lean proofs for the semantic definitions C_8 and enc themselves.

To connect the controls to the definitions that the statements actually use, each module proves the control values for C_8 and enc directly. Two general lemmas carry these proofs (Listing 2). If a horizontal ray from a non-barrier pixel avoids B, the pixel's component is infinite (`infinite_of_ray`). A non-barrier pixel whose four 4-neighbours lie in B forms a finite component (`isolated`). For the ring, every non-barrier pixel other than the centre has a free ray to the left or right. The centre is isolated. Hence exactly one component is enclosed. The block and diamond are handled the same way, and $C_8 = 1$ follows from explicit 8-paths. Combined with `decide` on χ, this checks the identity enc $= C_8 - \chi$ for the ring $(1 = 1 - 0)$, the block $(0 = 1 - 1)$ and the diamond $(1 = 1 - 0)$ on the semantic definitions.

Listing 2: Key lemmas behind the semantic controls (excerpt from the proposal modules).

```
/-- A horizontal ray of non-barrier pixels makes a component infinite. -/
theorem infinite_of_ray (B : Finset Pixel) (v : Pixel) (hv : v ∉ B) (d : ℤ) (hd : d = 1 ∨ d = -1)
    (hray : ∀ n : ℕ, (v.1 + d * n, v.2) ∉ B) :
    ¬ IsEnclosed ((complementGraph B).connectedComponentMk ⟨v, hv⟩) := by
  intro hfin
  have hreach : ∀ n : ℕ, (complementGraph B).Reachable ⟨v, hv⟩ ⟨(v.1 + d * n, v.2), hray n⟩ := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
      refine ih.trans (SimpleGraph.Adj.reachable ?_)
      simp only [complementGraph, SimpleGraph.comap_adj, SimpleGraph.induce, G4,
        SimpleGraph.fromRel_adj, Function.Embedding.coe_subtype, ne_eq, Prod.mk.injEq]
      push_cast
      rcases hd with rfl | rfl <;> refine ⟨by omega, Or.inl ?_⟩ <;> simp
  apply Set.infinite_of_injective_forall_mem (f := fun n : ℕ => (⟨(v.1 + d * n, v.2), hray n⟩ :
    ((B : Set Pixel)ᶜ : Set Pixel)))
  · intro m n h
    simp only [Subtype.mk.injEq, Prod.mk.injEq] at h
    rcases hd with rfl | rfl <;> omega
  · intro n
    exact (SimpleGraph.ConnectedComponent.mem_supp_iff _ _).2
      (SimpleGraph.ConnectedComponent.eq.2 (hreach n).symm)
  · exact hfin


/-- A pixel whose four 4-neighbours are all barrier is a finite component on its own. -/
theorem isolated (B : Finset Pixel) (h : Pixel) (hh : h ∉ B)
    (hn : ∀ u : Pixel, |h.1 - u.1| + |h.2 - u.2| = 1 → u ∈ B) :
    IsEnclosed ((complementGraph B).connectedComponentMk ⟨h, hh⟩) := by
  apply (Set.finite_singleton (⟨h, hh⟩ : ((B : Set Pixel)ᶜ : Set Pixel))).subset
  intro w hw
  rw [SimpleGraph.ConnectedComponent.mem_supp_iff, SimpleGraph.ConnectedComponent.eq] at hw
  obtain ⟨p⟩ := hw.symm
  cases p with
  | nil => rfl
  | @cons _ u _ hadj _ =>
    exfalso
    simp only [complementGraph, SimpleGraph.comap_adj, SimpleGraph.induce, G4,
      SimpleGraph.fromRel_adj, Function.Embedding.coe_subtype] at hadj
    obtain ⟨-, hr | hr⟩ := hadj
    · exact u.2 (hn _ hr)
    · exact u.2 (hn _ (by rw [abs_sub_comm h.1, abs_sub_comm h.2]; exact hr))

theorem enclosed_ring : enclosed ring = 1 := by
  refine enclosed_eq_one ring (1, 1) (by decide) (isolated _ _ _ ?_) ?_
```

```
· rintro ⟨x, y⟩ h
  have h' : |1 - x| + |1 - y| = 1 := h
  simp only [ring, Finset.mem_insert, Finset.mem_singleton, Prod.mk.injEq]
  rcases abs_cases (1 - x) with ⟨h1, h1'⟩ | ⟨h1, h1'⟩ <;>
  rcases abs_cases (1 - y) with ⟨h2, h2'⟩ | ⟨h2, h2'⟩ <;> omega
· rintro ⟨x, y⟩ hv hne
  simp only [ring, Finset.mem_insert, Finset.mem_singleton, Prod.mk.injEq, not_or] at hv
  simp only [ne_eq, Prod.mk.injEq] at hne
  by_cases hr : 2 < x ∨ y < 0 ∨ 2 < y
  · refine infinite_of_ray _ _ _ 1 (Or.inl rfl) (ray_right _ _ ?_)
    simp only [ring, Finset.mem_insert, Finset.mem_singleton, forall_eq_or_imp, forall_eq]
    omega
  · refine infinite_of_ray _ _ _ (-1) (Or.inr rfl) (ray_left _ _ ?_)
    simp only [ring, Finset.mem_insert, Finset.mem_singleton, forall_eq_or_imp, forall_eq]
    omega
```

9 Computational evidence for L2 and L3

The script `code/bridge_check.py` is independent of the Lean box model. It pads each raster by two pixels, labels the 4-components of the complement and the 8-components of B by depth-first search, and computes $V - E + F$ directly. It found no failures (`results/bridge_check.json`):

- L3 on all $2^{16} = 65{,}536$ subsets of a 4×4 box;
- L3 on 3,000 random rasters of side up to 12 with random density (seed 20261103);
- L2 on 10,496 insertions: each of the 256 configurations of the eight neighbours of p, alone and in 40 random contexts at Chebyshev distance ≥ 2 from p.

This is evidence that the proposed statements are true and that their conventions are the intended ones. It is not a proof.

10 Limitations

- The theorem concerns the combinatorial counts $V - E + F$ as defined in the sealed module. The development does not show that this number is the Euler characteristic of the topological space $K(B) \subset \mathbb{R}^2$. That identification is standard but is not formalised here, and L2/L3 are designed not to need it.
- Only the 8-connected weighting of the bit quads is treated. The 4-connected variant, with $+2Q_D$, is not formalised.
- L2 and L3 are proposed statements, not theorems. They have not been proved, so the `FaceCount` hypothesis of *Trees Cannot Cut* is not yet discharged.
- The bounded-box model used for the `decide` checks is not proved to agree with the semantic definitions. Only the four semantic control lemmas link the two.
- Checking all 256 neighbourhood configurations of L2 by `decide` was too slow for the kernel within this session. They were checked in Python only.
- L3 relates enclosed components to $C_8 - \chi$, not to the circuit rank of a corridor graph. A paper that uses L3 in place of `FaceCount` must state which raster-to-graph correspondence it uses. The 2×2 block shows that the two notions differ.

11 Reproducibility

The L1 proof file `proofs/P1.BitQuad.lean` and the script `code/bitquad_check.py` belong to the package of the 2026-10-28 run. It imports only `Mathlib` and the sealed module `ViridisStatements.P1.BitQuad` (sha256 `cd3cdeba48f6...8e23`), and it builds with toolchain `leanprover/lean4:v4.28.0` and Mathlib commit `8f9d9cff6bd728b17a24e163c9402775d9e6a365`. The check takes about 20 seconds. The brute-force script runs with the Python 3 standard library: `python3 code/bitquad_check.py results/bitquad_4x4.json`. No external data are used.

The proposals `proposals/P1.Insertion.lean` and `proposals/P1.Bridge.lean` each compile with `lean` against the same toolchain and Mathlib commit, in about 50 and 20 seconds respectively, including all `decide` checks and semantic control proofs. The evidence for L2/L3 is reproduced by `python3 -I code/bridge_check.py > results/bridge_check.json` (standard library only, about 15 seconds).

Changelog

- 2026-10-28 (P1-T001): first version of the paper. Lean proof of L1 (`P1.BitQuad`), proof architecture, brute-force check on the 4×4 box, and discussion of the role of L1 in L2/L3.
- 2026-11-03 (P1-T002): new title and abstract. Added the proposed statements `P1.Insertion` (L2) and `P1.Bridge` (L3) with their conventions, controls (box-model `decide` checks and Lean proofs of the control values for the semantic definitions), brute-force evidence for L2/L3, the reference to Rosenfeld (1970), and updated limitations and reproducibility.

References

[1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[2] Stephen B. Gray. Local properties of binary images in two dimensions. *IEEE Transactions on Computers*, C-20(5):551–561, 1971. doi: 10.1109/T-C.1971.223289.

[3] Justin D. Hart. Trees cannot cut: Circuit rank as the design variable reconciling habitat connectivity and fuel discontinuity in managed forest landscapes. Zenodo, Viridis LLC, 2026.

[4] T. Yung Kong and Azriel Rosenfeld. Digital topology: Introduction and survey. *Computer Vision, Graphics, and Image Processing*, 48(3):357–393, 1989. doi: 10.1016/0734-189X(89)90147-3.

[5] Azriel Rosenfeld. Connectivity in digital pictures. *Journal of the ACM*, 17(1):146–160, 1970. doi: 10.1145/321556.321570.

[6] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.